Week 07

클래스와 객체지향 설계

AI활용코딩연습(ZBE135)

이영호 교수 · 국립목포대학교 컴퓨터학부

지난주 돌아보기

```
try:
    record = validate_row(row)
except ValueError as e:
    errors.append({"line": line_no, "reason": str(e)})
else:
    valid.append(record)
```

- 예상한 오류만 잡고, **이유와 줄 번호를 기록**
- 처리 결과를 다시 입력으로 넣어 **재검증**

오늘의 목표

1. **상태와 동작을 묶는** 클래스가 필요한 이유를 이해한다
2. `class`, `__init__`, `self`, 메서드, `__str__`, `@property`를 쓴다
3. 메서드로만 상태를 바꿔 **잘못된 상태를 막는다** (캡슐화)
4. 여러 클래스가 협력하는 **도서 대출 관리 시스템**을 만든다
5. 상속 · dataclass를 맛보고, **클래스 변수 공유 버그**를 찾는다

수업 흐름

구간	시간	활동
도입 · 개념	20분	왜 클래스인가, 클래스 · 객체 · self
안내 실습	45분	Book 클래스, 특별 메서드, 도서 대출 시스템
독립 · 짝 실습	35분	시나리오 테스트, 상속 · dataclass
공유 · 점검	20분	클래스 변수 버그, 중간 실기 안내

이번 주 Colab 실습 열기

딕셔너리 + 함수의 한계

```
book = {"id": "B001", "title": "파이썬 첫걸음", "on_loan": False}

def borrow(book):
    book["on_loan"] = True

borrow(book)
borrow(book)                  # 이미 대출 중인데 또 빌려줌
book["on_loan"] = "네"         # 아무 값이나 넣을 수 있음
book["tittle"] = "오타"        # 오타 키가 새로 생겨도 모름
```

데이터와 규칙이 **따로** 있어서, 규칙을 **우회**할 수 있다

클래스와 객체

용어	비유	예
클래스	붕어빵 틀	`Book`
객체(인스턴스)	붕어빵	`Book("B001", "파이썬 첫걸음")`
속성	객체가 가진 데이터	`book.title`, `book.on_loan`
메서드	객체가 하는 동작	`book.borrow()`

> 클래스 = **데이터 + 그 데이터를 다루는 규칙**을 한곳에

첫 번째 클래스

```
class Book:
    def __init__(self, book_id, title, author="미상"):
        self.book_id = book_id
        self.title = title
        self.author = author
        self.on_loan = False

    def borrow(self):
        if self.on_loan:
            raise ValueError(f"이미 대출 중: {self.title}")
        self.on_loan = True

b1 = Book("B001", "파이썬 첫걸음", "김목포")
b1.borrow()
```

self는 "호출한 그 객체"

```
b1 = Book("B001", "파이썬 첫걸음")
b2 = Book("B002", "데이터 분석 입문")

b1.borrow()           # 실제로는 Book.borrow(b1)
print(b1.on_loan)     # True
print(b2.on_loan)     # False  ← 각자 자기 상태
b1.borrow()           # ValueError: 이미 대출 중
```

- `__init__`: 객체를 만들 때 **자동 실행**, 속성 준비
- 메서드의 첫 매개변수는 항상 `self`

예상해 보기: print(b1)

```
print(b1)
# <__main__.Book object at 0x7f...>   ← 알아보기 어렵다
```

메서드	언제	목적
`__str__`	`print(obj)`, `str(obj)`	사람이 읽기 좋게
`__repr__`	노트북 표시, 리스트 안	개발자용 정확한 표현

```
def __str__(self):
    return f"[{self.book_id}] {self.title} ({self.author}) - {self.status}"
```

@property: 계산되는 속성

```
class Book:
    ...
    @property
    def status(self):
        return "대출 중" if self.on_loan else "대출 가능"

b1.status        # 괄호 없이 읽는다 → '대출 중'
```

- 저장하지 않고 **필요할 때 계산**
- 다른 속성과 **항상 일치** (따로 갱신할 필요 없음)

실습 1 · 2 (Level 1 → Level 2)

Level 1 따라하기

- 딕셔너리 방식의 문제 재현
- `Book` 클래스: `__init__`, `borrow` / `give_back`, `ValueError`
- `__str__`, `__repr__`, `@property`

Level 2 바꾸기

- `loan_count` (누적 대출 횟수) 추가, `__str__` 에 표시
- 예: 대출 횟수 제한 규칙 추가해 보기

실습 3 (Level 3): 도서 대출 관리

규정 (가상)

- 회원당 최대 **3권**, 대출 기간 **14일**, 연체료 **하루 100원**
- 없는 회원·책 → `KeyError`, 규칙 위반 → `ValueError`

클래스	상태	책임
`Book`	id, 제목, 대출 여부	대출/반납 상태 바꾸기
`Member`	이름, 빌린 책 목록	한도 확인
`Library`	책 · 회원 · 대출 기록	규칙 조율, 연체 조회

책임 나누기: Member와 Library

```
class Member:
    MAX_BOOKS = 3                          # 클래스 변수: 공유 상수

    def __init__(self, name):
        self.name = name
        self.borrowed = []                 # 인스턴스 변수: 회원마다 따로

    def can_borrow(self):
        return len(self.borrowed) < self.MAX_BOOKS

class Library:
    LOAN_DAYS, FINE_PER_DAY = 14, 100

    def __init__(self):
        self.books, self.members, self.loans = {}, {}, {}
```

대출 메서드: 규칙 조율

```
def borrow(self, name, book_id, today):
    member = self.members[name]          # 없으면 KeyError
    book = self.books[book_id]           # 없으면 KeyError
    if not member.can_borrow():
        raise ValueError(f"{name}: 대출 한도 초과")
    book.borrow()                        # 이미 대출 중이면 Book이 거부
    member.borrowed.append(book_id)
    due = today + timedelta(days=self.LOAN_DAYS)
    self.loans[book_id] = (name, due)
    return due
```

각 객체가 **자기 규칙을 스스로** 지킨다 → `Library`는 순서만 조율

반납과 연체료

```
def give_back(self, book_id, today):
    if book_id not in self.loans:
        raise ValueError(f"대출 기록 없음: {book_id}")
    name, due = self.loans.pop(book_id)
    self.books[book_id].give_back()
    self.members[name].borrowed.remove(book_id)
    late_days = max(0, (today - due).days)
    return late_days * self.FINE_PER_DAY
```

> 상태가 **세 곳**(책 · 회원 · 대출 기록)에 있으므로 **함께** 갱신해야 한다

시나리오 테스트

```
def make_library():                          # 테스트마다 새 객체
    lib = Library()
    for i in range(1, 6):
        lib.add_book(Book(f"B00{i}", f"책{i}"))
    lib.join(Member("가")); lib.join(Member("나"))
    return lib

def test_fine_boundary():
    lib = make_library()
    d = date(2026, 9, 1)
    lib.borrow("가", "B001", d)
    assert lib.give_back("B001", d + timedelta(days=14)) == 0     # 당일
    lib.borrow("가", "B001", d)
    assert lib.give_back("B001", d + timedelta(days=15)) == 100   # 1일 연체
```

상속: 공통은 물려받고 다른 것만

```
class EBook(Book):
    def __init__(self, book_id, title, author="미상", licenses=3):
        super().__init__(book_id, title, author)
        self.licenses = licenses
        self.active = 0

    def borrow(self):                       # 오버라이딩
        if self.active >= self.licenses:
            raise ValueError("동시 대출 한도 초과")
        self.active += 1
```

다형성: `Library`는 `book.borrow()`만 부르면 된다 — 책이든 전자책이든

@dataclass

```python
from dataclasses import dataclass, field

@dataclass
class LoanRecord:
    book_id: str
    member: str
    due: date
    returned: bool = False
    notes: list = field(default_factory=list)   # 가변 기본값

r1 = LoanRecord("B001", "김민준", date(2026, 9, 15))
print(r1)        # LoanRecord(book_id='B001', member='김민준', ...)
```

`__init__` · `__repr__` · `__eq__` 자동 생성 — **데이터 중심 클래스**에

AI 코드: 클래스 변수 공유

```
class MemberAI:
    borrowed = []                  # 클래스 몸체의 리스트

    def __init__(self, name):
        self.name = name

    def add(self, book_id):
        self.borrowed.append(book_id)

m1, m2 = MemberAI("김민준"), MemberAI("이서연")
m1.add("B001")
print(m2.borrowed)                 # ['B001']  ← ?!
```

클래스 변수 vs 인스턴스 변수

	클래스 변수	인스턴스 변수
위치	클래스 몸체	`__init__` 안 `self.x = ...`
공유	**모든 객체가 하나를 공유**	객체마다 따로
용도	`MAX_BOOKS = 3` 같은 **상수**	이름, 빌린 목록 같은 **상태**

```
def test_members_have_separate_lists(cls):
    a, b = cls("가"), cls("나")
    a.add("B001")
    return b.borrowed == []
```

5주차 **가변 기본값**과 같은 뿌리: 리스트 하나를 여럿이 공유

AI에게 클래스를 요청할 때

```
회원(Member) 클래스를 만들어 주세요.
- 회원마다 따로 가져야 하는 리스트·딕셔너리는 __init__ 안에서 self로 만들 것
- 상태 변경은 메서드로만 하고, 규칙 위반은 ValueError로 알릴 것
- 두 회원 객체가 서로 영향을 주지 않는지 확인하는 테스트를 포함할 것
```

받은 코드에서 확인: 클래스 몸체에 `[]` · `{}`가 있는가? 상태가 여러 곳이면 **함께** 갱신하는가?

흔한 실수 모음

실수	증상	해결
메서드에 `self` 빠뜨림	`TypeError: takes 0 positional arguments`	`def borrow(self):`
`self.` 없이 속성 사용	`NameError`	`self.title`
클래스 몸체에 리스트	객체끼리 공유	`__init__`에서 생성
상태를 한 곳만 갱신	책은 반납, 회원 목록엔 남음	관련 상태 함께 갱신
테스트끼리 객체 공유	순서에 따라 결과가 바뀜	테스트마다 새 객체

이번 주 과제 (10점)

문제	내용	배점
1	`BankAccount` 입금 · 출금 · 기록 · `__str__`	2
2	`Rectangle` · `Square`(상속) · `Circle`, `total_area` 다형성	2
3	`TodoList` 번호 · 완료 · 마감 · `@property progress`	3
4	스터디룸 예약 시스템 (AI 협업)	3

문제 4: AI에게 **설계를 먼저** 묻고, 상태 두 곳(시스템 · 학생)을 함께 갱신하는지 검증

다음 주: 중간 실기 안내

- 범위: **1~7주차** (문자열 · 제어문 · 자료구조 · 함수 · 파일 · 예외 · 클래스)
- 형식: 성적 데이터를 다루는 **통합 문제 4개** + 1쪽 기술 메모
- 자동 확인 셀 + 코드 읽기 + **설명 능력**으로 평가
- AI 사용 범위는 다음 주 안내를 따름

> 준비: 1~7주차 **과제 노트북을 AI 없이 다시 풀어 보기**

자기 점검

1. `b1.borrow()`에서 `self`는 무엇인가?
2. 딕셔너리 대신 클래스를 쓰면 막을 수 있는 실수 두 가지는?
3. `MAX_BOOKS = 3`을 클래스 변수로 둔 이유는?
4. 반납할 때 함께 갱신해야 하는 상태 세 가지는?
5. 시나리오 테스트마다 새 `Library`를 만드는 이유는?

정리와 다음 주

오늘	핵심
클래스	상태 + 동작, `__init__`, `self`
캡슐화	메서드로만 상태 변경, 규칙 위반은 `ValueError`
협력	책임 나누기, 관련 상태 함께 갱신
상속	`super()`, 오버라이딩, 다형성, `@dataclass`
AI	클래스 변수 리스트 공유 → `__init__`에서 생성

다음 주 · Week 08 중간 실기: Python 문제 해결